

作业管理系统过程剖析

1 开发与调试概述

本系统开发过程中，我重点记录了三个真实调试案例。它们都不是事后手写错误页面，而是在系统运行、页面点击或文件上传过程中真实复现出来的：一是教师发布作业时异常课程编号导致后端报错；二是学生选课表单方法与 Flask 路由不一致导致 405；三是学生上传中文文件名附件后，教师端显示文件名丢失。

调试时使用的主要验证方式如下：

```
.venv\Scripts\python -m unittest discover -s tests -v  
python scripts\init_db.py --reset  
python app.py
```

其中单元测试用于回归验证，浏览器页面操作用于复现真实用户路径。下面按问题现象、排查过程、修复方法、修复后验证四个角度记录。

2 问题一：发布作业时课程编号异常导致服务器错误

2.1 问题现象

在教师发布作业功能调试时，我发现后端接口对表单中的课程编号缺少异常处理。正常页面上的课程下拉框一般只会提交数字编号，但如果浏览器重复提交旧页面、表单被异常修改，或前端传来的 `course_id` 不是数字，后端会直接执行 `int()` 转换并抛出异常，页面显示 500 错误。

Internal Server Error

The server encountered an internal error and was unable to complete your request. Either the server is overloaded or there is an error in the application.

图 1：修复前课程编号异常导致 500 错误

2.2 排查过程

根据错误日志继续追踪，异常位置定位在 `assignment_system/routes.py` 的教师发布作业路由中。原代码直接写成：

```
course_id = int(request.form.get("course_id", 0))
```

这行代码默认表单中的课程编号一定可以转成整数。实际测试时提交 `course_id=abc`，程序会抛出：

```
ValueError: invalid literal for int() with base 10: 'abc'
```

这个问题属于典型的表单输入校验不完整。虽然正常用户通过下拉框操作时不容易遇到，但后端不能完全依赖前端控件，仍然需要对收到的数据做校验。

2.3 修复方法

修复时在课程编号转换外层增加 `try/except`。当课程编号缺失或不是数字时，不再让 Flask 返回 500，而是向用户显示提示并回到教师工作台。

```
try:
    course_id = int(request.form.get("course_id", 0))
except (TypeError, ValueError):
    flash("请选择有效课程。")
    return redirect(url_for("teacher_dashboard"))
```

2.4 修复后效果

使用同样的异常表单再次提交后，系统不再出现 500 错误，而是回到教师工作台，并在顶部显示“请选择有效课程。”提示。

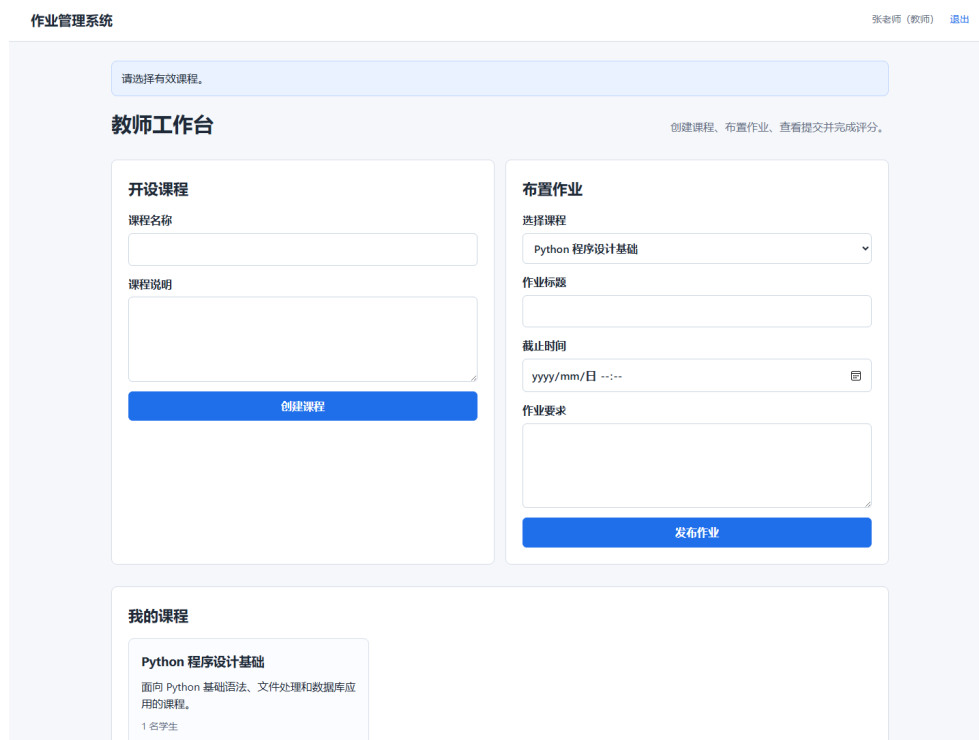


图 2：修复后显示有效课程提示

3 问题二：学生选课表单方法错误导致 405

3.1 问题现象

在联调学生加入课程功能时，我遇到过一次页面按钮能显示、但点击后没有完成选课的问题。真实复现步骤是：先用教师账号创建一门新课程，再切换到学生账号，在“可选课程”中点击“加入课程”。点击后浏览器没有回到学生工作台，而是进入 Flask 默认的 Method Not Allowed 页面。

Method Not Allowed
The method is not allowed for the requested URL.

图 3：修复前点击加入课程后出现 405

3.2 排查过程

先看页面 URL，错误发生在 `/student/courses/2/enroll?`。再回到代码中对照路由定义，发现后端路由只允许 POST：

```
@app.post("/student/courses/<int:course_id>/enroll")
@role_required("student")
def student_enroll_course(course_id):
    ...
```

但是模板中的表单方法写成了 GET：

```
<form method="get" action="{{ url_for('student_enroll_course',
                                         course_id=course.id) }}">
    <button type="submit">加入课程</button>
</form>
```

这就导致浏览器真实点击按钮时发送 GET 请求，而 Flask 端只接受 POST，于是返回 405。这个问题不是数据库问题，也不是权限问题，而是前端模板和后端路由方法没有对齐。

3.3 修复方法

修复方式是把学生选课表单改回 POST，使页面提交方式和 Flask 路由定义一致。

```
<form method="post" action="{% url_for('student_enroll_course',
                                         course_id=course.id) %}">

    <button type="submit">加入课程</button>

</form>
```

3.4 修复后效果

修复后使用同样流程重新测试：教师创建课程，学生点击“加入课程”。页面不再进入 405，而是回到学生工作台，顶部显示“选课成功。”，课程状态也变为“已加入”。

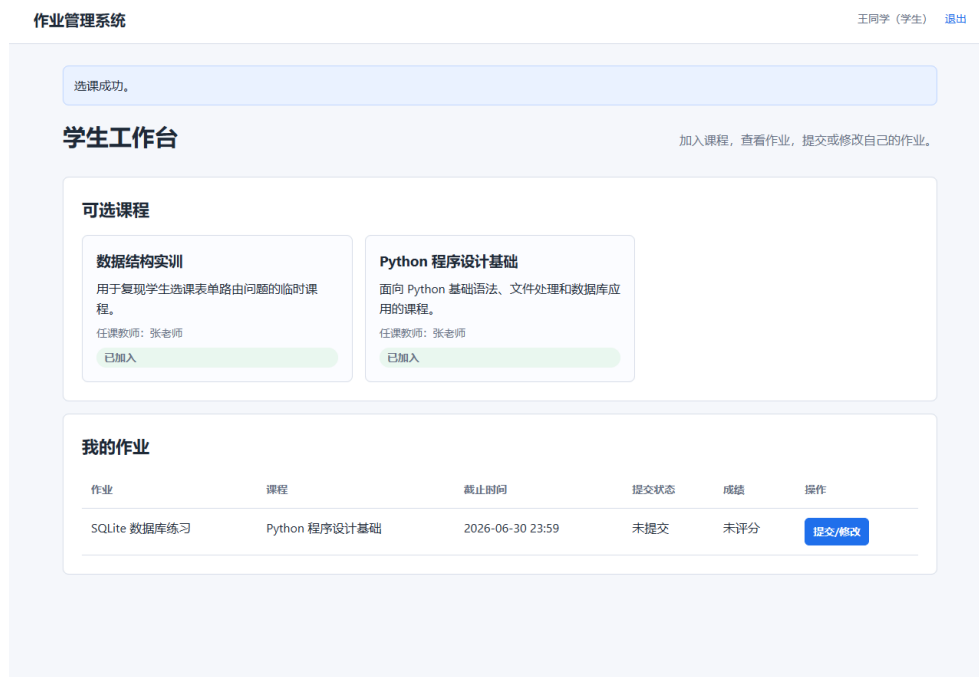


图 4：修复后点击加入课程成功

4 问题三：中文附件文件名被处理后丢失

4.1 问题现象

在调试学生提交附件功能时，我用学生页面真实上传了一个名为 数据库练习报告.docx 的文件。提交成功后切换到教师账号查看提交记录，发现文件列没有显示原来的中文文件名，而只剩下 docx。

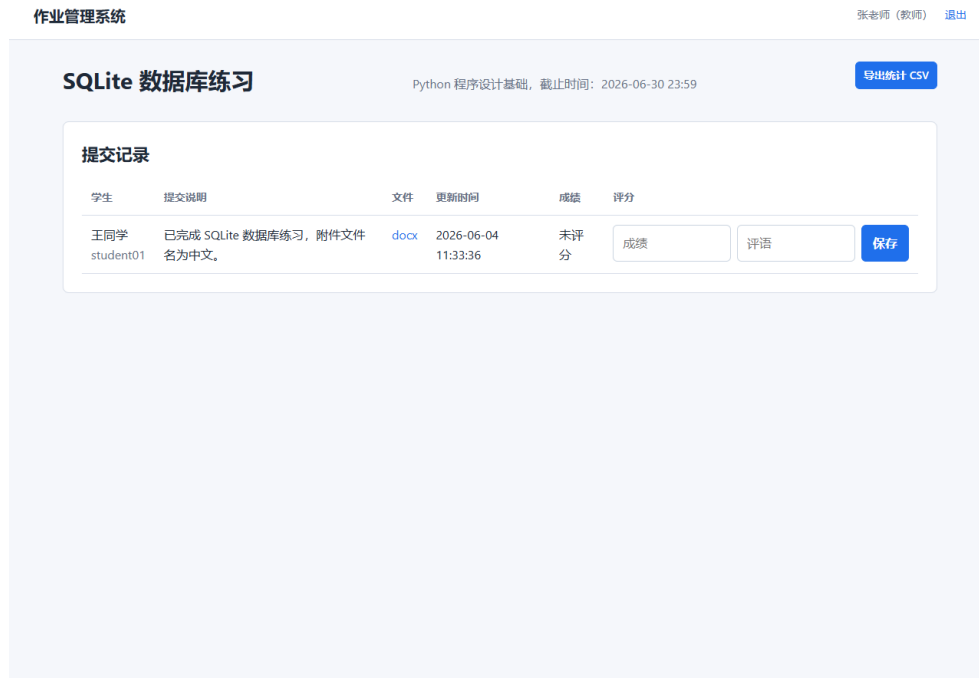


图 5：修复前中文附件名显示为 docx

4.2 排查过程

问题定位到 `SubmissionService.save()` 中的文件名处理逻辑。原代码把 `secure_filename()` 的结果同时当作页面展示名和服务器存储名：

```
if file_storage and file_storage.filename:
    filename = secure_filename(file_storage.filename) or "upload.dat"
    stored_filename = f"{uuid4().hex}_{filename}"
    file_storage.save(upload_dir() / stored_filename)
```

`secure_filename()` 的作用是生成适合服务器保存的安全文件名，它会去掉路径符号、空格和不安全字符。对于中文文件名，中文主体会被移除，例如 数据库练习报告.docx 会变成 docx。如果直接把这个结果显示给教师，用户看到的原文件名就丢失了。

4.3 修复方法

修复思路是把“用户看到的文件名”和“服务器存储的文件名”分开：页面展示和下载继续使用原始文件名；服务器实际保存时仍然使用安全文件名并加上随机编号。

```
def display_upload_filename(raw_filename):
    name = Path(raw_filename.replace("\\", "/")).name.strip()
    return name or "upload.dat"

def stored_upload_filename(display_name):
    safe_name = secure_filename(display_name)
    suffix = Path(display_name).suffix
    if suffix and safe_name == suffix.lstrip("."):
        safe_name = f"upload{suffix}"
    return f"{uuid4().hex}_{safe_name or 'upload.dat'}"

if file_storage and file_storage.filename:
    filename = display_upload_filename(file_storage.filename)
    stored_filename = stored_upload_filename(filename)
    file_storage.save(upload_dir() / stored_filename)
```

4.4 修复后效果

修复后再次通过学生页面上传同一个中文文件名附件。教师端提交记录中可以看到完整的 数据库练习报告.docx，服务器内部仍然使用随机安全文件名保存，避免同名覆盖和路径风险。



图 6: 修复后教师端保留中文附件名

5 自动化验证与经验总结

为了避免这些问题以后再次出现，我补充了自动化测试。测试覆盖了完整 workflows、异常课程编号、学生选课表单方法、中文附件文件名保留、教师注册审核等场景。最终测试结果如下：

```
test_admin_teacher_student_workflow ... ok
test_chinese_upload_filename_is_preserved ... ok
test_invalid_assignment_course_id_shows_message ... ok
test_pending_teacher_cannot_login_before_approval ... ok
test_student_enroll_form_uses_post_and_submits ... ok

Ran 5 tests in 3.363s
OK
```

问题类型	表现	处理方式
表单输入异常	页面返回 500，后端日志出现 ValueError	增加 try/except 和业务提示，并补充单元测试
表单方法错误	学生点击加入课程后进入 405 页面	对齐模板 method 与 Flask 路由方法，并测试 POST 提交
中文文件名丢失	教师端只看到 docx，原中文名消失	展示名和服务器存储名分离，保留原文件名下载
回归验证不足	修复后无法确认是否会复发	将关键异常输入和页面流程写入 tests 目录

通过这些调试可以看出，系统开发不能只关注“正常流程能跑通”。后端接口需要校验异常输入，模板和路由要保持一致，文件上传也要区分用户可见信息和服务器安全存储信息。最终修复后，系统在异常输入、页面点击和中文文件上传场景下都更加稳定。